Task 1:

We extended the LangNum grammar by using regular expressions to support binary, decimal and hexadecimal integers.
In the regex we made sure that both BINARY_LITERAL and HEX_LITERAL, start with the proper prefix, and including DECIMAL_LITERAL all three support negative value and underscore sbetween digits.

```
BINARY_LITERAL:          /-?0b[01]+(_?[01]+)*/
DECIMAL_LITERAL:          /-?[0-9]+(_?[0-9]+)*/
HEX_LITERAL:            /-?0x[0-9a-fA-F]+(_?[0-9a-fA-F]+)*/
```

The pattern also ensures that underscores cant appear at the beginning or the end, and makes sure the prefixes are case sensitive so 0B and 0X aren’t valid.
The num_literal transformer removes the underscore and converts each integer according to its appropriate base.
Binary with base 2, hex with 16 and decimal with 10. The string is sliced up to 2, since the prefixes are the same everytime, and up to 3 if its a negative too. what matters in the conversion is what comes after the sign and prefix.
The implementation was tested by writing 5 examples in progs/num named task1_1→5.odc, with the expected outcome and running the ./check.sh script which they all passed.